CS1010 Discrete Mathematics for Computer Science

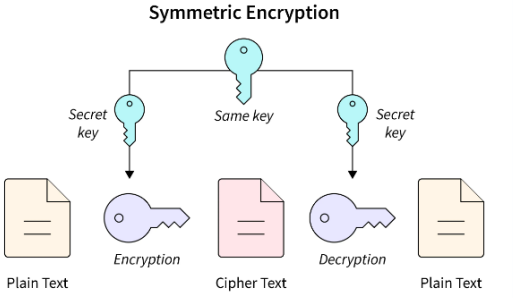
Shibam Ghosh

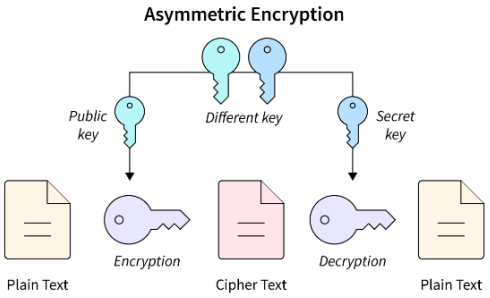
September 7, 2026

Department of CSE, IIT Hyderabad

Modern Cryptography

Symmetric and Asymmetric Key Cryptography





Modern Crypto Infrastructure

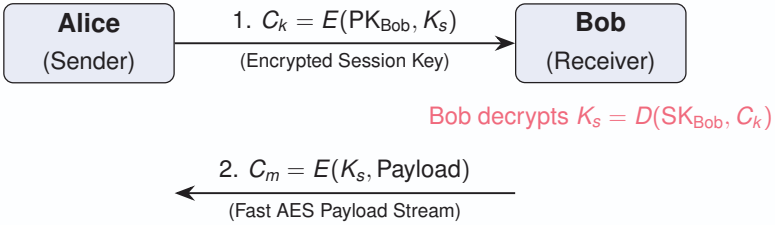
The Core Principle

Symmetric-key cryptography is ‘fast’, but need a pre-shared secret key. Use **Asymmetric** cryptography to exchange a short-lived secret key, then use **Symmetric Encryption** to encrypt the actual payload data.

Modern Crypto Infrastructure

The Core Principle

Symmetric-key cryptography is 'fast', but need a pre-shared secret key. Use **Asymmetric** cryptography to exchange a short-lived secret key, then use **Symmetric Encryption** to encrypt the actual payload data.



Real-World Usage

- **HTTPS / TLS Handshake:** The foundation of secure web browsing.
- **SSH (Secure Shell):** Uses RSA/ECDSA to negotiate AES keys for terminal access.
- **PGP / GPG Email Security:** Encrypts large email attachments with symmetric keys while wrapping the key asymmetrically.

Key Takeaways

- Asymmetric crypto provides **key establishment**
- Symmetric crypto provides **efficient data transfer**

Symmetric Key Cryptosystem

Formal Definition: Symmetric Key Cryptosystem (Cipher)

A symmetric key cryptosystem is five-tuple:

$$(\mathscr{P}, \mathscr{C}, \mathscr{K}, \mathscr{E}, \mathscr{D})$$

- $\mathscr{P}$ is a finite set of possible **plaintexts**.
- $\mathscr{C}$ is a finite set of possible **ciphertexts**.
- $\mathscr{K}$ is a finite set of possible **keys**.
- $\mathscr{E}$ and $\mathscr{D}$ are set of deterministic algorithms

For each $K \in \mathscr{K}$, $\exists$ an encryption rule $E_K \in \mathscr{E}$ and a decryption rule $D_K \in \mathscr{D}$:

$$E_K : \mathscr{P} \to \mathscr{C} \quad \text{and} \quad D_K : \mathscr{C} \to \mathscr{P} \text{ such that } D_k(E_k(P)) = P \quad \forall P \in \mathscr{P}$$

Formal Definition: Symmetric Key Cryptosystem (Cipher)

A symmetric key cryptosystem is five-tuple:

$$(\mathcal{P}, \mathcal{C}, \mathcal{K}, \mathcal{E}, \mathcal{D})$$

- $\mathcal{P}$ is a finite set of possible **plaintexts**.
- $\mathcal{C}$ is a finite set of possible **ciphertexts**.
- $\mathcal{K}$ is a finite set of possible **keys**.
- $\mathcal{E}$ and $\mathcal{D}$ are set of deterministic algorithms

For each $K \in \mathcal{K}$, $\exists$ an encryption rule $E_K \in \mathcal{E}$ and a decryption rule $D_K \in \mathcal{D}$:

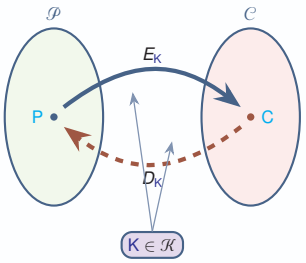
$$E_K : \mathcal{P} \to \mathcal{C} \quad \text{and} \quad D_K : \mathcal{C} \to \mathcal{P} \text{ such that } D_k(E_k(P)) = P \quad \forall P \in \mathcal{P}$$

Formal Definition: Symmetric Key Cryptosystem (Cipher)

A symmetric key cryptosystem is five-tuple:

$$(\mathcal{P}, \mathcal{C}, \mathcal{K}, \mathcal{E}, \mathcal{D})$$

- $\mathcal{P}$ is a finite set of possible **plaintexts**.
- $\mathcal{C}$ is a finite set of possible **ciphertexts**.
- $\mathcal{K}$ is a finite set of possible **keys**.
- $\mathcal{E}$ and $\mathcal{D}$ are set of deterministic algorithms



For each $K \in \mathcal{K}$, $\exists$ an encryption rule $E_K \in \mathcal{E}$ and a decryption rule $D_K \in \mathcal{D}$:

$$E_K : \mathcal{P} \to \mathcal{C} \quad \text{and} \quad D_K : \mathcal{C} \to \mathcal{P} \text{ such that } D_k(E_k(P)) = P \quad \forall P \in \mathcal{P}$$

Kerckhoffs' Principle

Kerckhoffs' Note

Both algorithm sets $\mathcal{E}$ and $\mathcal{D}$ are publicly known

- Assume the attacker knows every structural feature and limit of the cipher
- We do not rely on keeping the design of our algorithm a secret
- Must remain secure even if the adversary steals the source code

The Only Secret is the **Key** denoted as K

Kerckhoffs' Principle

Kerckhoffs' Note

Both algorithm sets $\mathcal{E}$ and $\mathcal{D}$ are publicly known

- Assume the attacker knows every structural feature and limit of the cipher
- We do not rely on keeping the design of our algorithm a secret
- Must remain secure even if the adversary steals the source code

The Only Secret is the **Key** denoted as K

Kerckhoffs' Principle

Kerckhoffs' Note

Both algorithm sets $\mathcal{E}$ and $\mathcal{D}$ are publicly known

- Assume the attacker knows every structural feature and limit of the cipher
- We do not rely on keeping the design of our algorithm a secret
- Must remain secure even if the adversary steals the source code

The Only Secret is the **Key** denoted as K

Cipher Design Criteria

1. Must be **efficient** to encrypt and decrypt whenever the correct key is provided
2. Must be computationally **infeasible** to:
   - ▷ Decrypt data without knowing the key
   - ▷ Extract the key
   - ▷ Extrapolate any statistical information on the plaintext

One-Way Functions

These functions are public algorithms that are computationally **efficient** to compute forward, but exceptionally **difficult** to invert (without the key).

Cipher Design Criteria

1. Must be **efficient** to encrypt and decrypt whenever the correct key is provided
2. Must be computationally **infeasible** to:
   - ▷ Decrypt data without knowing the key
   - ▷ Extract the key
   - ▷ Extrapolate any statistical information on the plaintext

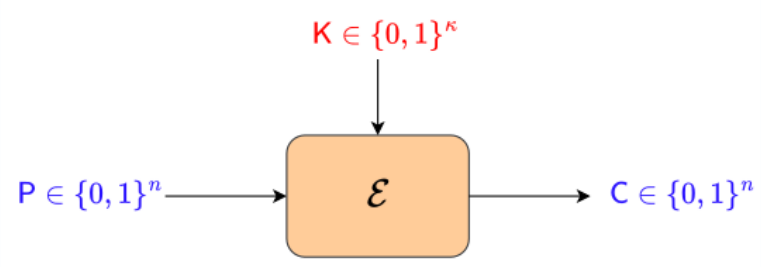
One-Way Functions

These functions are public algorithms that are computationally **efficient** to compute forward, but exceptionally **difficult** to invert (without the key).

Symmetric Key Cryptosystem

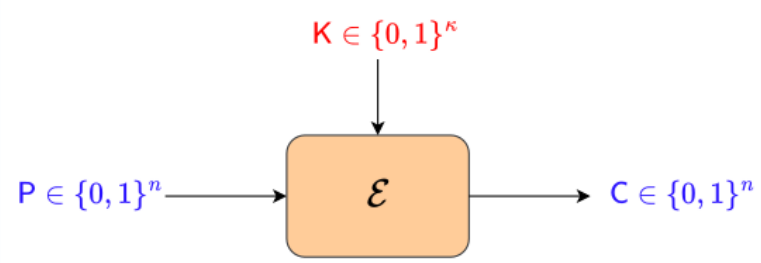
Block Cipher

Block Cipher



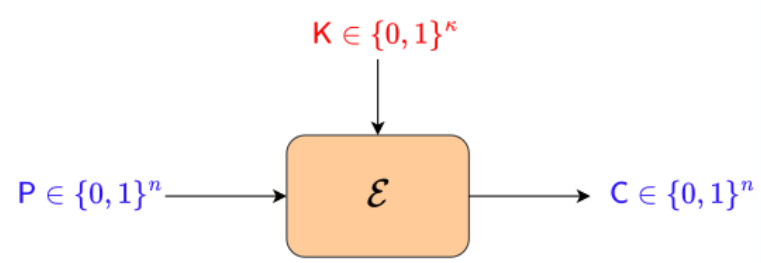
- $E : \{0,1\}^n \times \{0,1\}^\kappa \rightarrow \{0,1\}^n$ s.t. $C = E(P, K)$
- $E_K : \{0,1\}^n \rightarrow \{0,1\}^n$ s.t. $C = E_K(P)$
- Map n-bit plaintext blocks to n-bit ciphertext blocks (n = block length)
- The inverse mapping is the decryption function, $P = D_K(C) = E_K^{-1}(C)$

Block Cipher



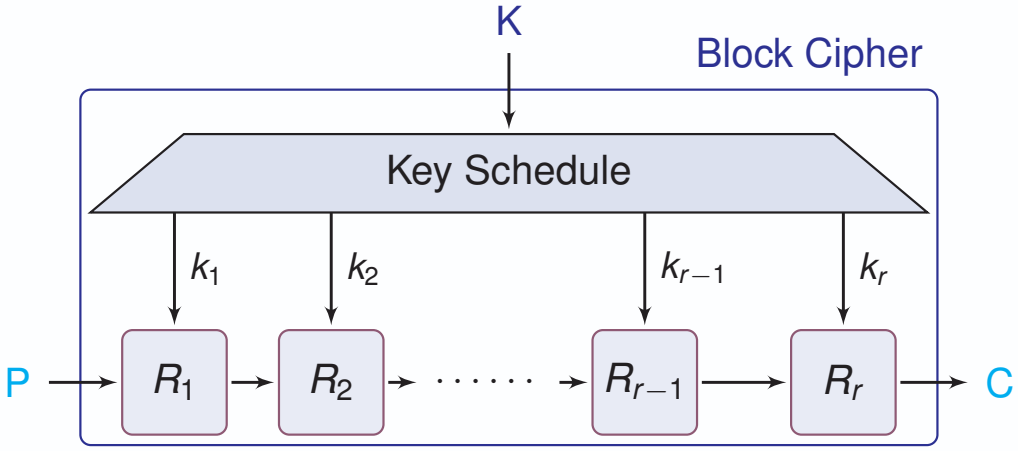
- $E : \{0,1\}^n \times \{0,1\}^\kappa \rightarrow \{0,1\}^n$ s.t. $C = E(P, K)$
- $E_K : \{0,1\}^n \rightarrow \{0,1\}^n$ s.t. $C = E_K(P)$
- Map n-bit plaintext blocks to n-bit ciphertext blocks (n = block length)
- The inverse mapping is the decryption function, $P = D_K(C) = E_K^{-1}(C)$

Block Cipher



- $E : \{0,1\}^n \times \{0,1\}^\kappa \rightarrow \{0,1\}^n$ s.t. $C = E(P, K)$
- $E_K : \{0,1\}^n \rightarrow \{0,1\}^n$ s.t. $C = E_K(P)$
- Map n-bit plaintext blocks to n-bit ciphertext blocks (n = block length)
- The inverse mapping is the decryption function, $P = D_K(C) = E_K^{-1}(C)$

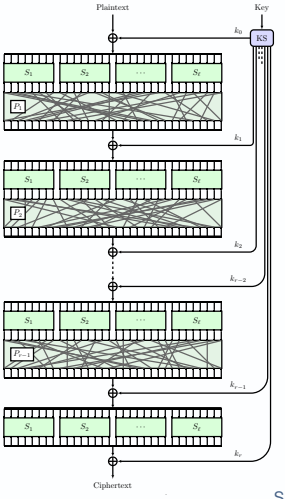
Iterated Block Cipher



Confusion and Diffusion Paradigm by Claude Shannon

- *Confusion* property aims to make the relationship between the ciphertext and the (plaintext, key) complex
- *Diffusion* property ensures that each bit of the plaintext and the key influences many bits of the ciphertext

Substitution Permutatio Network



Public Key Cryptography

Why Public Key Cryptography?

How do they securely share that key in the first place?

- In traditional (*symmetric*) cryptography, Alice and Bob must share the **exact same secret key** to encrypt and decrypt messages.

Public Key / Asymmetric Cryptography

A cryptographic system that uses a **mathematically linked pair of keys**:

Public Key: Shared openly with the world. Anyone can use it to **encrypt**.

Private Key: Kept secret by the owner. Only it can be used to **decrypt**.

Examples: RSA: Introduced in 1976 by Rivest, Shamir, and Adleman (RSA). It is the backbone of secure identity validation in **HTTPS** (web browsing), **SSH** (secure command-line access), and **PGP** (encrypted email).

The RSA Cryptosystem

Key Generation:

1. Select two large distinct primes, p and q (each ≥ 200 digits).
2. Compute the modulus $n = pq$ and the totient $\phi(n) = (p-1)(q-1)$.
3. Choose an exponent e such that $\gcd(e, \phi(n)) = 1$.
4. Compute d, such that $de \equiv 1 \pmod{(p-1)(q-1)}$.
5. **Public Key:** (n, e) **Private Key:** d

Encryption: Given a message integer M ($M < n$): $\mathbf{C = M^e \pmod{n}}$

Decryption: Given a ciphertext integer C: $\mathbf{M = C^d \pmod{n}}$

It's Security relies on the difficulty of factoring $n = pq$.

The RSA Cryptosystem

Key Generation:

1. Select two large distinct primes, p and q (each ≥ 200 digits).
2. Compute the modulus $n = pq$ and the totient $\phi(n) = (p-1)(q-1)$.
3. Choose an exponent e such that $\gcd(e, \phi(n)) = 1$.
4. Compute d, such that $de \equiv 1 \pmod{(p-1)(q-1)}$.
5. **Public Key:** (n, e) **Private Key:** d

Encryption: Given a message integer M ($M < n$): $\mathbf{C = M^e \pmod{n}}$

Decryption: Given a ciphertext integer C: $\mathbf{M = C^d \pmod{n}}$

It's Security relies on the difficulty of factoring $n = pq$.

The RSA Cryptosystem

Key Generation:

1. Select two large distinct primes, p and q (each ≥ 200 digits).
2. Compute the modulus $n = pq$ and the totient $\phi(n) = (p-1)(q-1)$.
3. Choose an exponent e such that $\gcd(e, \phi(n)) = 1$.
4. Compute d, such that $de \equiv 1 \pmod{(p-1)(q-1)}$.
5. **Public Key:** (n, e) **Private Key:** d

Encryption: Given a message integer M ($M < n$): $\mathbf{C = M^e \pmod{n}}$

Decryption: Given a ciphertext integer C: $\mathbf{M = C^d \pmod{n}}$

It's Security relies on the difficulty of factoring $n = pq$.

The RSA Cryptosystem

Key Generation:

1. Select two large distinct primes, p and q (each ≥ 200 digits).
2. Compute the modulus $n = pq$ and the totient $\phi(n) = (p-1)(q-1)$.
3. Choose an exponent e such that $\gcd(e, \phi(n)) = 1$.
4. Compute d, such that $de \equiv 1 \pmod{(p-1)(q-1)}$.
5. **Public Key:** (n, e) **Private Key:** d

Encryption: Given a message integer M ($M < n$): $\mathbf{C = M^e \pmod{n}}$

Decryption: Given a ciphertext integer C: $\mathbf{M = C^d \pmod{n}}$

It's Security relies on the difficulty of factoring $n = pq$.

The RSA Cryptosystem

Key Generation:

1. Select two large distinct primes, p and q (each ≥ 200 digits).
2. Compute the modulus $n = pq$ and the totient $\phi(n) = (p-1)(q-1)$.
3. Choose an exponent e such that $\gcd(e, \phi(n)) = 1$.
4. Compute d, such that $de \equiv 1 \pmod{(p-1)(q-1)}$.
5. **Public Key:** (n, e) **Private Key:** d

Encryption: Given a message integer M ($M < n$): $\mathbf{C = M^e \pmod{n}}$

Decryption: Given a ciphertext integer C: $\mathbf{M = C^d \pmod{n}}$

It's Security relies on the difficulty of factoring $n = pq$.

RSA Example

1. **Key Generation:**
   - Let $p = 3$, $q = 11$, so $n = 3 \cdot 11 = \mathbf{33}$
   - $\phi(n) = (3-1)(11-1) = \mathbf{20}$
   - Choose $\mathbf{e} = \mathbf{7}$ (since $\gcd(7, 20) = 1$)
   - Find d: $d \cdot 7 \equiv 1 \pmod{20} \implies \mathbf{d} = \mathbf{3}$ *(Since* $3 \cdot 7 = 21 \equiv 1$*)*
   - Public Key: $(7, 33)$ and Private Key: 3
2. **Encrypt Message** $M = 2$**:**

$$C = 2^7 \pmod{33}$$

$$C = 128 \pmod{33} = \mathbf{29}$$

3. **Decrypt Ciphertext** $C = 29$**:**

$$M = 29^3 \pmod{33}$$

$$M = 24389 \pmod{33} = \mathbf{2}$$

(Successfully recovered original M!)

Correctness of RSA

The Goal: Decrypting a ciphertext C always yields the original message M:

$$(\mathbf{M^e})^{\mathbf{d}} \equiv \mathbf{M^{ed}} \equiv \mathbf{M} \pmod{\mathbf{n}}$$

1. $ed \equiv 1 \pmod{\phi(n)} \implies \exists k \in \mathbb{Z}$ such that $ed = k \cdot \phi(n) + 1 = k(p-1)(q-1) + 1$
2. $M^{ed} = M^{k(p-1)(q-1)+1} = (\mathbf{M^{(p-1)}})^{\mathbf{k(q-1)}} \cdot \mathbf{M}$
3. Assuming $\gcd(M, p) = 1$, Fermat's Little Theorem tells us that $M^{p-1} \equiv 1 \pmod{p}$.

$$\therefore M^{ed} \equiv (1)^{k(q-1)} \cdot M \equiv M \pmod{p}$$

4. By applying the exact same logic using modulo q, we get: $M^{ed} \equiv M \pmod{q}$

Conclusion via Chinese Remainder Theorem (CRT):
Because p and q are distinct prime numbers, $\gcd(p, q) = 1$. Since $M^{ed} \equiv M \pmod{p}$ and $M^{ed} \equiv M \pmod{q}$, it must be divisible by their product $n = pq$.

$$\text{Hence, } \mathbf{M^{ed}} \equiv \mathbf{M} \pmod{\mathbf{n}} \quad \checkmark$$

Correctness of RSA

The Goal: Decrypting a ciphertext C always yields the original message M:

$$(\mathbf{M^e})^{\mathbf{d}} \equiv \mathbf{M^{ed}} \equiv \mathbf{M} \pmod{\mathbf{n}}$$

1. $ed \equiv 1 \pmod{\phi(n)} \implies \exists k \in \mathbb{Z}$ such that $ed = k \cdot \phi(n) + 1 = k(p-1)(q-1) + 1$
2. $M^{ed} = M^{k(p-1)(q-1)+1} = (\mathbf{M^{(p-1)}})^{\mathbf{k(q-1)}} \cdot \mathbf{M}$
3. Assuming $\gcd(M, p) = 1$, Fermat's Little Theorem tells us that $M^{p-1} \equiv 1 \pmod{p}$.

$$\therefore M^{ed} \equiv (1)^{k(q-1)} \cdot M \equiv M \pmod{p}$$

4. By applying the exact same logic using modulo q, we get: $M^{ed} \equiv M \pmod{q}$

Conclusion via Chinese Remainder Theorem (CRT):
Because p and q are distinct prime numbers, $\gcd(p, q) = 1$. Since $M^{ed} \equiv M \pmod{p}$ and $M^{ed} \equiv M \pmod{q}$, it must be divisible by their product $n = pq$.

$$\text{Hence, } \mathbf{M^{ed}} \equiv \mathbf{M} \pmod{\mathbf{n}} \quad \checkmark$$

Correctness of RSA

The Goal: Decrypting a ciphertext C always yields the original message M:

$$(\mathbf{M^e})^{\mathbf{d}} \equiv \mathbf{M^{ed}} \equiv \mathbf{M} \pmod{\mathbf{n}}$$

1. $ed \equiv 1 \pmod{\phi(n)} \implies \exists k \in \mathbb{Z}$ such that $ed = k \cdot \phi(n) + 1 = k(p-1)(q-1) + 1$
2. $M^{ed} = M^{k(p-1)(q-1)+1} = (\mathbf{M^{(p-1)}})^{\mathbf{k(q-1)}} \cdot \mathbf{M}$
3. Assuming $\gcd(M, p) = 1$, Fermat's Little Theorem tells us that $M^{p-1} \equiv 1 \pmod{p}$.

$$\therefore M^{ed} \equiv (1)^{k(q-1)} \cdot M \equiv M \pmod{p}$$

4. By applying the exact same logic using modulo q, we get: $M^{ed} \equiv M \pmod{q}$

Conclusion via Chinese Remainder Theorem (CRT):

Because p and q are distinct prime numbers, $\gcd(p, q) = 1$. Since $M^{ed} \equiv M \pmod{p}$ and $M^{ed} \equiv M \pmod{q}$, it must be divisible by their product $n = pq$.

Hence, $\mathbf{M^{ed}} \equiv \mathbf{M} \pmod{\mathbf{n}}$ ✓

Correctness of RSA

The Goal: Decrypting a ciphertext C always yields the original message M:

$$(\mathbf{M^e})^{\mathbf{d}} \equiv \mathbf{M^{ed}} \equiv \mathbf{M} \pmod{\mathbf{n}}$$

1. $ed \equiv 1 \pmod{\phi(n)} \implies \exists k \in \mathbb{Z}$ such that $ed = k \cdot \phi(n) + 1 = k(p-1)(q-1) + 1$
2. $M^{ed} = M^{k(p-1)(q-1)+1} = (\mathbf{M^{(p-1)}})^{\mathbf{k(q-1)}} \cdot \mathbf{M}$
3. Assuming $\gcd(M, p) = 1$, Fermat's Little Theorem tells us that $M^{p-1} \equiv 1 \pmod{p}$.
$$\therefore M^{ed} \equiv (1)^{k(q-1)} \cdot M \equiv M \pmod{p}$$
4. By applying the exact same logic using modulo q, we get: $M^{ed} \equiv M \pmod{q}$

Conclusion via Chinese Remainder Theorem (CRT):
Because p and q are distinct prime numbers, $\gcd(p, q) = 1$. Since $M^{ed} \equiv M \pmod{p}$ and $M^{ed} \equiv M \pmod{q}$, it must be divisible by their product $n = pq$.

$$\text{Hence, } \mathbf{M^{ed}} \equiv \mathbf{M} \pmod{\mathbf{n}} \quad \checkmark$$

RSA Decryption When $\gcd(M, n) > 1$

Consider all the setups as RSA. Suppose that

$$C \equiv M^e \pmod{pq}.$$

Show that, even if $\gcd(M, pq) > 1$:

$$C^d \equiv M \pmod{pq}$$

Since d is the inverse of e modulo $(p-1)(q-1)$, there exists an integer k such that

$$ed = 1 + k(p-1)(q-1).$$

- We prove the congruence separately modulo p and modulo q.
- Then we combine the two results using the **Chinese Remainder Theorem**.

RSA Decryption When $\gcd(M, n) > 1$

Consider all the setups as RSA. Suppose that

$$C \equiv M^e \pmod{pq}.$$

Show that, even if $\gcd(M, pq) > 1$:

$$C^d \equiv M \pmod{pq}$$

Since d is the inverse of e modulo $(p-1)(q-1)$, there exists an integer k such that

$$ed = 1 + k(p-1)(q-1).$$

- We prove the congruence separately modulo p and modulo q.
- Then we combine the two results using the **Chinese Remainder Theorem**.

RSA Decryption When $\gcd(M, n) > 1$

Consider all the setups as RSA. Suppose that

$$C \equiv M^e \pmod{pq}.$$

Show that, even if $\gcd(M, pq) > 1$:

$$C^d \equiv M \pmod{pq}$$

Since d is the inverse of e modulo $(p-1)(q-1)$, there exists an integer k such that

$$ed = 1 + k(p-1)(q-1).$$

- We prove the congruence separately modulo p and modulo q.
- Then we combine the two results using the **Chinese Remainder Theorem**.

Proof Modulo p and Modulo q

Since $C \equiv M^e \pmod{pq}$, we have

$$C^d \equiv M^{ed} \pmod{p}.$$

Using $ed = 1 + k(p-1)(q-1)$,

$$M^{ed} = M^{1+k(p-1)(q-1)} = M\left(M^{p-1}\right)^{k(q-1)}.$$

Now consider two cases modulo p:

- $p \nmid M$: By Fermat's little theorem, $M^{p-1} \equiv 1 \pmod{p}$. Hence
$$M^{ed} \equiv M \pmod{p}.$$
- $p \mid M$: Then $M \equiv 0 \pmod{p}$, and therefore $M^{ed} \equiv 0 \equiv M \pmod{p}$.

Thus, in *both* cases, $C^d \equiv M \pmod{p}$.

Proof Modulo p and Modulo q

Since $C \equiv M^e \pmod{pq}$, we have

$$C^d \equiv M^{ed} \pmod{p}.$$

Using $ed = 1 + k(p-1)(q-1)$,

$$M^{ed} = M^{1+k(p-1)(q-1)} = M\left(M^{p-1}\right)^{k(q-1)}.$$

Now consider two cases modulo p:

- $p \nmid M$: By Fermat's little theorem, $M^{p-1} \equiv 1 \pmod{p}$. Hence
$$M^{ed} \equiv M \pmod{p}.$$
- $p \mid M$: Then $M \equiv 0 \pmod{p}$, and therefore $M^{ed} \equiv 0 \equiv M \pmod{p}$.

Thus, in *both* cases, $C^d \equiv M \pmod{p}$.

Proof Modulo p and Modulo q

Since $C \equiv M^e \pmod{pq}$, we have

$$C^d \equiv M^{ed} \pmod{p}.$$

Using $ed = 1 + k(p-1)(q-1)$,

$$M^{ed} = M^{1+k(p-1)(q-1)} = M\left(M^{p-1}\right)^{k(q-1)}.$$

Now consider two cases modulo p:

- $p \nmid M$**:** By Fermat's little theorem, $M^{p-1} \equiv 1 \pmod{p}$. Hence
$$M^{ed} \equiv M \pmod{p}.$$
- $p \mid M$**:** Then $M \equiv 0 \pmod{p}$, and therefore $M^{ed} \equiv 0 \equiv M \pmod{p}$.

Thus, in *both* cases, $C^d \equiv M \pmod{p}$.

Proof Modulo p and Modulo q

Since $C \equiv M^e \pmod{pq}$, we have

$$C^d \equiv M^{ed} \pmod{p}.$$

Using $ed = 1 + k(p-1)(q-1)$,

$$M^{ed} = M^{1+k(p-1)(q-1)} = M\left(M^{p-1}\right)^{k(q-1)}.$$

Now consider two cases modulo p:

- $p \nmid M$: By Fermat's little theorem, $M^{p-1} \equiv 1 \pmod{p}$. Hence
$$M^{ed} \equiv M \pmod{p}.$$
- $p \mid M$: Then $M \equiv 0 \pmod{p}$, and therefore $M^{ed} \equiv 0 \equiv M \pmod{p}$.

Thus, in *both* cases, $C^d \equiv M \pmod{p}$.

Proof Modulo p and Modulo q

Since $C \equiv M^e \pmod{pq}$, we have

$$C^d \equiv M^{ed} \pmod{p}.$$

Using $ed = 1 + k(p-1)(q-1)$,

$$M^{ed} = M^{1+k(p-1)(q-1)} = M\left(M^{p-1}\right)^{k(q-1)}.$$

Now consider two cases modulo p:

- $p \nmid M$: By Fermat's little theorem, $M^{p-1} \equiv 1 \pmod{p}$. Hence
$$M^{ed} \equiv M \pmod{p}.$$

- $p \mid M$: Then $M \equiv 0 \pmod{p}$, and therefore $M^{ed} \equiv 0 \equiv M \pmod{p}$.

Thus, in *both* cases, $C^d \equiv M \pmod{p}$.

Applying the Chinese Remainder Theorem

By exactly the same argument modulo q, we obtain

$$C^d \equiv M \pmod{q}.$$

Therefore,

$$p \mid (C^d - M) \qquad \text{and} \qquad q \mid (C^d - M).$$

Since $\gcd(p, q) = 1$., by the **Chinese Remainder Theorem**,

$$pq \mid (C^d - M).$$

$$C^d \equiv M \pmod{pq}$$

Thus, RSA decryption works for **every** $M \in \mathbb{Z}_n$.

Applying the Chinese Remainder Theorem

By exactly the same argument modulo q, we obtain

$$C^d \equiv M \pmod{q}.$$

Therefore,

$$p \mid (C^d - M) \qquad \text{and} \qquad q \mid (C^d - M).$$

Since $\gcd(p, q) = 1$., by the **Chinese Remainder Theorem**,

$$pq \mid (C^d - M).$$

$$C^d \equiv M \pmod{pq}$$

Thus, RSA decryption works for **every** $M \in \mathbb{Z}_n$.

Applying the Chinese Remainder Theorem

By exactly the same argument modulo q, we obtain

$$C^d \equiv M \pmod{q}.$$

Therefore,

$$p \mid (C^d - M) \qquad \text{and} \qquad q \mid (C^d - M).$$

Since $\gcd(p, q) = 1$., by the **Chinese Remainder Theorem**,

$$pq \mid (C^d - M).$$

$$C^d \equiv M \pmod{pq}$$

Thus, RSA decryption works for **every** $M \in \mathbb{Z}_n$.

Applying the Chinese Remainder Theorem

By exactly the same argument modulo q, we obtain

$$C^d \equiv M \pmod{q}.$$

Therefore,

$$p \mid (C^d - M) \qquad \text{and} \qquad q \mid (C^d - M).$$

Since $\gcd(p, q) = 1$., by the **Chinese Remainder Theorem**,

$$pq \mid (C^d - M).$$

$$\boxed{C^d \equiv M \pmod{pq}}$$

Thus, RSA decryption works for **every** $M \in \mathbb{Z}_n$.

Public Key Cryptography

Discussion on RSA

Why Is RSA Suitable for Public-Key Cryptography?

- RSA relies on an important asymmetry between two computational tasks:

Generating large primes is efficient, while factoring the product of two large primes is believed to be computationally hard.

- **Key Generation: Easy**
  Choose two large primes p and q, and compute

$$n = pq, \qquad \varphi(n) = (p-1)(q-1).$$

- Since $gcd(e, \varphi(n)) = 1$, we can efficiently compute $d \equiv e^{-1} \pmod{\varphi(n)}$.
- The public key is (n, e), while d is kept secret.

Why Is RSA Suitable for Public-Key Cryptography?

- RSA relies on an important asymmetry between two computational tasks:

> **Generating large primes is efficient, while factoring the product of two large primes is believed to be computationally hard.**

- **Key Generation: Easy**
  Choose two large primes p and q, and compute

$$n = pq, \qquad \varphi(n) = (p-1)(q-1).$$

- Since $gcd(e, \varphi(n)) = 1$, we can efficiently compute $d \equiv e^{-1} \pmod{\varphi(n)}$.
- The public key is (n, e), while d is kept secret.

The RSA Trapdoor: Easy with p, q, Hard without Them

Legitimate User

- Knows the factorization $n = pq$.
- Therefore, they can compute $\varphi(n) = (p-1)(q-1)$,
- and thus efficiently obtain $d = e^{-1} \pmod{\varphi(n)}$.

Knowing p and q gives the trapdoor.

Adversary

- **Only** knows n, e.
- To compute the private exponent d, they would need $\varphi(n)$.
- $\varphi(n)$ is available when the factors p and q are known.

Factoring n is believed to be hard.

RSA is useful because the public information allows anyone to encrypt, but recovering the secret require solving a computationally difficult problem.

The RSA Trapdoor: Easy with p, q, Hard without Them

Legitimate User

- Knows the factorization $n = pq$.
- Therefore, they can compute $\varphi(n) = (p-1)(q-1)$,
- and thus efficiently obtain $d = e^{-1} \pmod{\varphi(n)}$.

Knowing p and q gives the trapdoor.

Adversary

- **Only** knows n, e.
- To compute the private exponent d, they would need $\varphi(n)$.
- $\varphi(n)$ is available when the factors p and q are known.

Factoring n is believed to be hard.

RSA is useful because the public information allows anyone to encrypt, but recovering the secret require solving a computationally difficult problem.

The Computational Assumption Behind RSA

- Therefore, RSA relies on the fact that:

> Multiplication is easy, but factoring is believed to be hard.

- This assumption is computational rather than information-theoretic: a future improvement in factoring algorithms could require larger RSA parameters.
- There is also a long-term concern:

> A sufficiently powerful quantum computer running Shor's algorithm could efficiently factor RSA moduli and break RSA.

The future is Post-Quantum Cryptography (PQC).

The Computational Assumption Behind RSA

- Therefore, RSA relies on the fact that:

> Multiplication is easy, but factoring is believed to be hard.

- This assumption is computational rather than information-theoretic: a future improvement in factoring algorithms could require larger RSA parameters.
- There is also a long-term concern:

> A sufficiently powerful quantum computer running Shor's algorithm could efficiently factor RSA moduli and break RSA.

The future is Post-Quantum Cryptography (PQC).

The Computational Assumption Behind RSA

- Therefore, RSA relies on the fact that:

  Multiplication is easy, but factoring is believed to be hard.

- This assumption is computational rather than information-theoretic: a future improvement in factoring algorithms could require larger RSA parameters.
  - There is also a long-term concern:

  A sufficiently powerful quantum computer running Shor's algorithm could efficiently factor RSA moduli and break RSA.

The future is Post-Quantum Cryptography (PQC).

Thank You for your attention!

Any questions?